

# Reading StimGen 2 sample files

The .sgb format, with examples in C, Python and Julia

Michele Giugliano

Companion note to the StimGen 2 specification — October 2026

## Contents

|          |                                                                  |          |
|----------|------------------------------------------------------------------|----------|
| <b>1</b> | <b>Purpose</b>                                                   | <b>1</b> |
| <b>2</b> | <b>The format at a glance</b>                                    | <b>2</b> |
| <b>3</b> | <b>Python</b>                                                    | <b>3</b> |
| 3.1      | With numpy . . . . .                                             | 3        |
| 3.2      | With the standard library only . . . . .                         | 4        |
| 3.3      | Large files, integrity, regeneration . . . . .                   | 5        |
| <b>4</b> | <b>C</b>                                                         | <b>6</b> |
| <b>5</b> | <b>Julia</b>                                                     | <b>8</b> |
| 5.1      | With Base only . . . . .                                         | 8        |
| 5.2      | With the JSON package, a memory map and a digest check . . . . . | 9        |
| <b>6</b> | <b>Summary of pitfalls</b>                                       | <b>9</b> |

## 1 Purpose

The renderer `sg` writes the samples of a stimulus to a `.sgb` file (StimGen binary). This note shows how to read such a file from three languages, with programs short enough to copy into an analysis script:

- **Python**, with `numpy`, or with the standard library only;
- **C** (C99), with no library at all;
- **Julia**, with Base only, or with the `JSON` package.

Every program below is complete. They are tested automatically against files written by `sg` (`tests/test_access.py` extracts each code block of this note, runs it, and compares the results of all languages). The format itself is defined in Chapter 15 of the specification.

All examples read the file `pair.sgb`, rendered from this stimulus:

```
sg 2 stimulus
rate 10kHz
seed 42
channel pre unit=pA {
```

```

    100ms  dc(0)
    @train
    200ms  pulses(amp=2000, rate=20Hz, width=1ms)
}
channel post unit=pA { 300ms dc(-50) + ou(0, 5, 2ms) }
digital cam { 300ms pulses(1, 100Hz, width=1ms) }

```

with `sg render pair.sg`. Each program prints the same summary, so the results of the three languages can be compared line by line.

## 2 The format at a glance

Table 1: Layout of a `.sgb` file ( $J$  channels of  $N$  samples).

| Offset (bytes) | Size (bytes) | Content                                                                      |
|----------------|--------------|------------------------------------------------------------------------------|
| 0              | 8            | magic: the characters S, G, B and five zero bytes                            |
| 8              | 8            | $L$ , the length of the header: unsigned 64-bit integer, little-endian       |
| 16             | $L$          | header: a JSON object, UTF-8, padded with spaces to make $L$ a multiple of 8 |
| $16 + L$       | $8JN$        | samples: IEEE 754 doubles, little-endian, channel after channel              |

The samples of channel 1 come first ( $N$  values), then those of channel 2, and so on, in the order of the `channels` list of the header. Sample  $k$  of every channel is at time  $t_k = k/\text{rate}$ . The header of `pair.sgb` is (lines wrapped, the description shortened):

```

{"version": "2", "rate": 10000, "samples": 3000,
 "channels": [{"name": "pre", "unit": "pA", "rest": 0, "digital": false},
               {"name": "post", "unit": "pA", "rest": 0, "digital": false},
               {"name": "cam", "unit": "1", "rest": 0, "digital": true}],
 "markers": [{"name": "pre.train", "sample": 1000}],
 "provenance": {"spec_version": "2",
 "implementation": "sg 0.2 (StimGen 2 reference renderer)",
 "repro_level": "B",
 "description": "### file: pair.sg\nsg 2 stimulus\nrate 10kHz\n...",
 "canonical_sha256": "d3e70e8d...", "files": [],
 "rate": 10000, "master_seed": "42", "trial": null,
 "samples_sha256": "e4529201...",
 "warnings": [], "clipping": null, "timestamp": "2026-10-02T22:41:14Z"}}

```

The fields that analysis programs usually need are:

Table 2: Main fields of the header.

| Field                                  | Meaning                                              |
|----------------------------------------|------------------------------------------------------|
| <code>rate</code>                      | sampling rate in Hz                                  |
| <code>samples</code>                   | number of samples $N$ per channel                    |
| <code>channels[j].name, .unit</code>   | name and unit of channel $j$ ("1": no unit)          |
| <code>channels[j].rest</code>          | value held before and after the stimulus             |
| <code>channels[j].digital</code>       | <b>true</b> for a digital line (samples are 0 or 1)  |
| <code>markers[m].name, .sample</code>  | labelled instants, as sample indices                 |
| <code>provenance.master_seed</code>    | the seed used, as a <b>string</b> (a 64-bit integer) |
| <code>provenance.samples_sha256</code> | SHA-256 of the sample bytes, to check the file       |
| <code>provenance.trial</code>          | for protocol trials: indices and variable values     |

Because  $L$  is a multiple of 8, the samples start on an 8-byte boundary: they can be memory-mapped as an array of doubles directly (Julia's `Mmap` requires this alignment; so does an aligned C pointer).

Three details matter in every language:

1. **Byte order.** Header length and samples are little-endian. On the usual processors (x86, ARM) this is the native order, but a portable reader states it explicitly, as the programs below do.
2. **Layout.** The file stores channel after channel. In row-major languages (C, numpy) the natural array is `x[channel][sample]`; in Julia, which is column-major, it is `X[sample, channel]`.
3. **The seed is a string.** Many JSON readers store numbers as doubles, which cannot hold every 64-bit integer, so `master_seed` is written as a string. Convert it with an integer parser, not as a float.

## 3 Python

### 3.1 With numpy

```
import json, sys
import numpy as np

def read_sgb(path):
    """Return (header, x): header as a dict, x[channel, sample] in channel units."""
    with open(path, "rb") as f:
        b = f.read()
        if b[:8] != b"SGB\0\0\0\0\0\0":
            raise ValueError(path + " is not a .sgb file")
        L = int.from_bytes(b[8:16], "little") # header length
        h = json.loads(b[16:16 + L])
        x = np.frombuffer(b, dtype="<f8", offset=16 + L) # little-endian doubles
        return h, x.reshape(len(h["channels"]), h["samples"])

path = sys.argv[1] if len(sys.argv) > 1 else "pair.sgb"
```

```

h, x = read_sgb(path)
t = np.arange(h["samples"]) / h["rate"] # time axis in s
print("%s: %d channel(s) x %d samples at %g Hz"
      % (path, x.shape[0], x.shape[1], h["rate"]))
for c, row in zip(h["channels"], x):
    print("%s [%s]: min %.6g, max %.6g, mean %.6g"
          % (c["name"], c["unit"], row.min(), row.max(), row.mean()))
for m in h["markers"]:
    print("marker %s at %.6g s" % (m["name"], m["sample"] / h["rate"]))
print("master seed", int(h["provenance"]["master_seed"]))

```

To use one channel by name, `x[[c["name"] for c in h["channels"]].index("post")]`. The small tool `tools/sgplot.py` of the project contains the same reader (load) and plots a file with `python3 tools/sgplot.py pair.sgb`.

### 3.2 With the standard library only

Where numpy is not available (for example on an acquisition computer with a minimal Python), the module `array` reads the doubles:

```

import array, json, sys

def read_sgb(path):
    """Return (header, x): x is a list of channels, each an array('d')."""
    with open(path, "rb") as f:
        b = f.read()
    if b[:8] != b"SGB\0\0\0\0\0\0\0":
        raise ValueError(path + " is not a .sgb file")
    L = int.from_bytes(b[8:16], "little")
    h = json.loads(b[16:16 + L])
    a = array.array("d", b[16 + L:])
    if sys.byteorder == "big": # the file is little-endian
        a.byteswap()
    N = h["samples"]
    return h, [a[j * N:(j + 1) * N] for j in range(len(h["channels"]))]

path = sys.argv[1] if len(sys.argv) > 1 else "pair.sgb"
h, x = read_sgb(path)
print("%s: %d channel(s) x %d samples at %g Hz"
      % (path, len(x), h["samples"], h["rate"]))
for c, row in zip(h["channels"], x):
    print("%s [%s]: min %.6g, max %.6g, mean %.6g"
          % (c["name"], c["unit"], min(row), max(row), sum(row) / len(row)))
for m in h["markers"]:
    print("marker %s at %.6g s" % (m["name"], m["sample"] / h["rate"]))
print("master seed", int(h["provenance"]["master_seed"]))

```

### 3.3 Large files, integrity, regeneration

A ten-minute recording at 20 kHz with eight channels is about 770 MB. A memory map reads only the parts that are used. The same program checks the file against the digest stored in its header, and regenerates it with `sg` from the provenance record alone:

```
import hashlib, json, os, subprocess, sys, tempfile
import numpy as np

def header(path):
    with open(path, "rb") as f:
        if f.read(8) != b"SGB\0\0\0\0\0\0":
            raise ValueError(path + " is not a .sgb file")
        L = int.from_bytes(f.read(8), "little")
        return json.loads(f.read(L)), 16 + L

def memmap(path):
    """Samples as a read-only memory map, x[channel, sample]."""
    h, offset = header(path)
    return h, np.memmap(path, dtype="<f8", mode="r", offset=offset,
                        shape=(len(h["channels"]), h["samples"]))

def verify(path):
    """True if the sample bytes match provenance.samples_sha256."""
    h, offset = header(path)
    d = hashlib.sha256()
    with open(path, "rb") as f:
        f.seek(offset)
        for block in iter(lambda: f.read(1 << 20), b''):
            d.update(block)
    return d.hexdigest() == h["provenance"]["samples_sha256"]

def regenerate(path, sg="sg"):
    """Render the description, rate and seed of the header again with sg."""
    h, _ = header(path)
    p = h["provenance"]
    with tempfile.TemporaryDirectory() as d:
        src, out = os.path.join(d, "x.sg"), os.path.join(d, "x.sgb")
        with open(src, "w") as f:
            # the description, without "### file:" lines
            f.write("".join(l for l in p["description"].splitlines(True)
                            if not l.startswith("### file:")))
        subprocess.run([sg, "render", "-q", "-r", "%.17gHz" % p["rate"],
                        "-s", p["master_seed"], "-o", out, src], check=True)
    return header(out)[0]["provenance"]["samples_sha256"] == p["samples_sha256"]

path = sys.argv[1] if len(sys.argv) > 1 else "pair.sgb"
h, x = memmap(path)
```

```
print("memory map:", x.shape, "post mean %.6g" % x[1].mean())
print("digest ok:", verify(path))
print("regenerated identically:", regenerate(path, os.environ.get("SG", "sg")))
```

The regeneration works for a single stimulus whose used files are all in the description (they are). For a file that reads data with `file(...)`, the data file must also be present; its digest is in `provenance.files`.

## 4 C

The program below needs only the C standard library. It reads the header as a string and extracts the few fields it needs with a minimal scanner, which relies on the order in which `sg` writes the top-level fields (`rate`, `samples`, `channels`, `markers`, then `provenance`). For a complete header, use a JSON library such as `cJSON` or `jansson` on the string `json`.

```
/* read_sgb.c -- read a StimGen 2 .sgb file.  cc -std=c99 -o read_sgb read_sgb.c */
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* Little-endian 64-bit integer from 8 bytes, independent of the machine. */
static uint64_t le64(const unsigned char *p)
{
    uint64_t v = 0;
    for (int i = 7; i >= 0; i--) v = (v << 8) | p[i];
    return v;
}

/* Number following "key": at or after position from (first occurrence). */
static double json_number(const char *from, const char *key)
{
    char pat[64];
    snprintf(pat, sizeof pat, "\"%s\": ", key);
    const char *p = strstr(from, pat);
    return p ? strtod(p + strlen(pat), NULL) : -1;
}

/* Copy the string value following "key": at p into out; return the
 * position after it, or NULL if key does not occur before 'end'. */
static const char *json_string(const char *p, const char *end, const char *key,
                              char *out, size_t n)
{
    char pat[64];
    snprintf(pat, sizeof pat, "\"%s\": \"", key);
    p = strstr(p, pat);
    if (!p || p > end) return NULL;
    p += strlen(pat);
    size_t k = strcspn(p, "\"");
    snprintf(out, n, "%.s", (int)k, p);
    return p + k;
}
```

```

}

int main(int argc, char **argv)
{
    const char *path = argc > 1 ? argv[1] : "pair.sgb";
    FILE *f = fopen(path, "rb");
    unsigned char head[16];
    if (!f || fread(head, 1, 16, f) != 16 || memcmp(head, "SGB\0\0\0\0\0", 8)) {
        fprintf(stderr, "%s: not a .sgb file\n", path);
        return 1;
    }
    uint64_t L = le64(head + 8);           /* header length */
    char *json = malloc(L + 1);
    if (fread(json, 1, L, f) != L) return 1;
    json[L] = 0;

    double rate = json_number(json, "rate");
    long N = (long)json_number(json, "samples");
    const char *chans = strstr(json, "\"channels\"");
    const char *marks = strstr(json, "\"markers\"");
    const char *prov = strstr(json, "\"provenance\"");

    /* Channel names and units, in file order. */
    char name[16][64], unit[16][16];
    int J = 0;
    for (const char *p = chans; J < 16 &&
         (p = json_string(p, marks, "name", name[J], sizeof name[J])); J++)
        p = json_string(p, marks, "unit", unit[J], sizeof unit[J]);

    /* Samples: J blocks of N little-endian doubles. */
    size_t nb = (size_t)J * (size_t)N * 8;
    unsigned char *raw = malloc(nb);
    if (fread(raw, 1, nb, f) != nb) {
        fprintf(stderr, "%s: truncated\n", path);
        return 1;
    }
    fclose(f);
    double *x = malloc((size_t)J * (size_t)N * sizeof *x); /* x[j*N + k] */
    for (size_t i = 0; i < (size_t)J * (size_t)N; i++) {
        uint64_t bits = le64(raw + 8 * i);
        memcpy(&x[i], &bits, 8);           /* IEEE 754 bits */
    }

    printf("%s: %d channel(s) x %ld samples at %g Hz\n", path, J, N, rate);
    for (int j = 0; j < J; j++) {
        const double *c = x + (size_t)j * N;
        double lo = c[0], hi = c[0], sum = 0;
        for (long k = 0; k < N; k++) {
            if (c[k] < lo) lo = c[k];
            if (c[k] > hi) hi = c[k];
            sum += c[k];
        }
    }
}

```

```

        printf("%s [%s]: min %.6g, max %.6g, mean %.6g\n",
               name[j], unit[j], lo, hi, sum / N);
    }
    char mname[64];
    const char *p = marks;
    while ((p = json_string(p, prov, "name", mname, sizeof mname)))
        printf("marker %s at %.6g s\n", mname, json_number(p, "sample") / rate);
    char seed[32];
    json_string(prov, json + L, "master_seed", seed, sizeof seed);
    printf("master seed %llu\n", strtoull(seed, NULL, 10)); /* exact 64-bit */
    free(json); free(raw); free(x);
    return 0;
}

```

Two points of the program are worth copying into any C reader. The bytes of every sample are assembled into a 64-bit integer explicitly (`le64`) and then copied into a double with `memcpy`, which is portable and does not depend on the byte order or the alignment of the machine. And the master seed is read with `strtoull`, so that all 64 bits are kept.

## 5 Julia

### 5.1 With Base only

Julia reads the file in a few lines. Since Julia stores arrays by columns, the natural shape is `X[sample, channel]`: column `j` is channel `j`, with no copy. Without a JSON package, the fields needed are taken from the header with regular expressions.

*# read\_sgb.jl -- read a StimGen 2 .sgb file with Base Julia only.*

```

function read_sgb(path)
    b = read(path)
    b[1:8] == [0x53, 0x47, 0x42, 0, 0, 0, 0, 0] || error("$path is not a .sgb file")
    L = Int(ltoh(reinterpret(UInt64, b[9:16])[1])) # header length
    header = String(b[17:16+L])
    rate = parse(Float64, match(r"\\"rate\\": ([0-9.eE+-]+)", header)[1])
    N = parse(Int, match(r"\\"samples\\": (\\d+)", header)[1])
    chans = [(m[1], m[2]) for m in
        eachmatch(r"\\"name\\": \"([^\"]*)\\", \\"unit\\": \"([^\"]*)\\", header)]
    marks = [(m[1], parse(Int, m[2])) for m in
        eachmatch(r"\\"name\\": \"([^\"]*)\\", \\"sample\\": (\\d+)\\", header)]
    X = reshape(ltoh.(reinterpret(Float64, b[17+L:end])), N, length(chans))
    return (; header, rate, N, chans, marks, X)
end

path = isempty(ARGS) ? "pair.sgb" : ARGS[1]
s = read_sgb(path)
t = (0:s.N-1) ./ s.rate # time axis in s
J = length(s.chans)
println("$path: $J channel(s) x $(s.N) samples at $(Int(s.rate)) Hz")
using Printf
for (j, (name, unit)) in enumerate(s.chans)

```

```

    c = view(s.X, :, j)
    @printf("%s [%s]: min %.6g, max %.6g, mean %.6g\n", name, unit,
            minimum(c), maximum(c), sum(c) / length(c))
end
for (name, k) in s.marks
    @printf("marker %s at %.6g s\n", name, k / s.rate)
end
seed = parse{UInt64, match(r "\"master_seed\": \"(\d+)\"", s.header)[1]}
println("master seed ", seed)

```

@printf with %.6g is used so that the output can be compared with that of the C and Python programs.

## 5.2 With the JSON package, a memory map and a digest check

With the JSON package (import Pkg; Pkg.add("JSON")) the whole header becomes a dictionary. Mmap and SHA are standard libraries.

```

# sgb_tools.jl -- header with JSON.jl, samples by memory map, digest check.
using JSON, Mmap, SHA

function open_sgb(path)
    io = open(path)
    read(io, 8) == [0x53, 0x47, 0x42, 0, 0, 0, 0, 0] || error("not a .sgb file")
    L = Int(ltoh(read(io, UInt64)))
    h = JSON.parse(String(read(io, L)))
    J, N = length(h["channels"]), h["samples"]
    # Little-endian machines (x86, ARM) map the file directly; otherwise copy.
    X = ENDIAN_BOM == 0x04030201 ? Mmap.mmap(io, Matrix{Float64}, (N, J), 16 + L) :
        ltoh.(Mmap.mmap(io, Matrix{Float64}, (N, J), 16 + L))
    return h, X, 16 + L
end

function verify(path)
    h, X, offset = open_sgb(path)
    data = open(io -> (seek(io, offset); read(io)), path)
    return bytes2hex(sha256(data)) == h["provenance"]["samples_sha256"]
end

path = isempty(ARGS) ? "pair.sgb" : ARGS[1]
h, X, _ = open_sgb(path)
names = [c["name"] for c in h["channels"]]
post = X[:, findfirst(=="post"), names]
println("channels: ", join(names, ", "), "; rate ", h["rate"], " Hz")
println("post mean ", round(sum(post) / length(post), sigdigits = 6))
println("digest ok: ", verify(path))

```

## 6 Summary of pitfalls

Table 3: Common mistakes when reading `.sgb` files.

| Pitfall                                      | Remedy                                                                                                                       |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| reading the samples in the native byte order | state little-endian: " <code>&lt;f8</code> " (numpy), <code>ltoh</code> (Julia), <code>le64</code> (C)                       |
| transposed channels                          | file order is channel after channel: <code>x[channel, sample]</code> row-major, <code>X[sample, channel]</code> column-major |
| a seed that changes after reading            | <code>master_seed</code> is a string; parse it as a 64-bit integer                                                           |
| a truncated file                             | check that <code>8JN</code> bytes follow the header; check <code>samples_sha256</code>                                       |
| markers in seconds                           | markers are sample indices: divide by <code>rate</code>                                                                      |
| digital channels                             | stored as doubles 0.0 and 1.0, with <code>"digital": true</code>                                                             |